Execution Plan for Read Intent

Ilia Gogotchuri (2026/03/21)

Plan Overview

This plan spans 12 weeks from March 23 to June 14. The first 8 weeks will be used to build the core of the application and services, and the last 4 weeks are planned for deployment, polishing, testing, and miscellaneous tasks. Since 12 weeks isn't a long time for such a project for a solo engineer, I am reserving some features as "extras" for the last 4 weeks. I will complete those if the plan goes along faster than I expect and I have time at the end. The timeline seems optimistic, but I feel confident, since I have done my research and developed small PoCs for parts of the system.

I have split the project into epics and laid them out on the 12 week timeline.

Epics

1) Project setup and infrastructure for development

In this epic, I will set up my project and development environment to work more efficiently and safely. This includes:

- Github repository - Monorepo setup for UI, multiple services, and infra code
- Docker Compose setup for development with supporting services (Postgres, Redis, Kratos)
- Setting up shared type-safe contract generation with (OpenAPI/ConnectRPC) for the Go BFF Server and the Dart UI client.
- Basic PostgreSQL configuration and setup for future use (Permissions, databases, roles)
- Go and Flutter project initializations
- Documentation describing how things are tied together and flows for setup and everyday use.

2) Auth - basic password

- Set up Ory Kratos configuration for email password authentication, email verification with code, and password reset with code on verified email
- Setting up the SMTP mailing provider to actually deliver transactional emails
- Go BFF routing of the Kratos endpoints and middlewares for session authentication with the session token
- Flutter project routing, auth state, authenticated requests, and the screens for login and registration.

3) Article Scraping Service

This is one of the main challenges and unknowns for this plan. I will work in three phases to achieve a minimum viable version and progress toward a somewhat polished service.

- Python project setup
- Trafilatura setup for scraping and cleaning content (This will be the core of the service and hopefully handle a lot of the websites by itself)
- Nodriver headless scraping setup (This is used when sites have unreasonable bot restrictions and highly dynamic pages - Trafilatura isn't able to actually download the DOM and extract the content from it) - We will gather the fully rendered web page with Nodriver and pass it to Trafilatura for downstream processing.
- Dockerizing the service with dependencies and adding it to the Docker Compose setup

- Redis Stream event subscription for the Python service
- Redis Stream setup on the Go BFF side to pass the event to this service

4) Phoneme Generation Service

Here, we will use Kokoro's Python library and PyTorch to run the Misaki phoneme generation service, with espeak-ng as a fallback, since it has a much larger dictionary of word pronunciations.

The main reason this is separated as a service is the different needs and characteristics of computational and network resources. Later on, if we need to move services to different machines for scaling or other purposes, the scraping service will need more RAM and CPU, and likely will benefit from having more than a single server running it, where the phoneme generation will benefit a lot from a server with a GPU.

- Python project setup with Redis Stream (I can use the previous project as a template here)
- Dockerization with the right requirements
- Setting up text chunking and phoneme generation
- Tying everything together by pulling cleaned articles from the Redis Stream and putting the generated phonemes back

5) On-device TTS

Kokoro and ONNX Runtime integration with Flutter, performance testing, and tweaking. Some parts are split into the audio player epic.

- Picking up the right dependencies and setting up a solid asynchronous, non-blocking IO foundation here is important. The application must remain smooth and not be disrupted by the inference.
- NPU-GPU-CPU optimizations. This will allow the TTS to run faster and more smoothly on a dedicated Neural Processing Unit if the device has one; otherwise, fallback to GPU and then CPU in that order.
- Generating and caching audio for playback and seeking

6) Article reader and audio player

This is a two-part epic, as the title suggests.

The audio player component is closely coupled to the TTS.

For example, we will need to synchronize the already-generated "buffered" audio state, and there are many edge cases that are difficult to fine-tune with real-time-generated audio:

- What happens when the user changes the playback speed?
- And what if they follow that action with another action, like seeking forward or backward?
- What if the audio they are seeking hasn't already been buffered or is buffered at a lower speed?

Since this is the case, and I am not particularly interested in developing a full-fledged real-time-generated audio player in Dart (In the scope of this project at least), I am likely going to skip some features and keep the audio player restricted to the following features:

- Play/Pause
- Seek on the timeline, up to the buffered part
- Timed jumps backward and forward X seconds
- If I implement the playback speed, it will be a global setting or set up sometime before the audio generation

Ignoring (but would like to see developed in the future):

- Seeking into the unbuffered timeline
- Changing the playback speed on the fly

The second component of this epic is the content reader. With Trafilatura, we are going to scrape the content with basic HTML formatting or in markdown (haven't decided yet), and this content will need to be rendered and displayed to the user in our app. The reader will have minimal player controls at the bottom, with an option to open the full player in full-screen mode.

7) Article Inbox and CRUD

Note: I am going to ignore background notifications delivery with Firebase, since it is a hassle to set up and not highly important for this project. Since the idea is to read what and when you choose, not when a notification is received.

Note 2: I had the OpenAPI spec mentioned in the proposal, but after further research, I like the ergonomics of Protobufs + ConnectRPC more and will go with that for API.

- Implementation of the Go BFF side API for article CRUD and Inbox endpoints.
- Streaming HTTP pipe for event delivery when the app is in the foreground for notifying job statuses (Article queued, article parsed, phonemes generated) and fetching the relevant endpoints accordingly.
- Endpoint wiring to the UI
- Screens for the list of finished, archived, and inbox articles
- Operations on the articles (Delete, Archive, Mark as read)
- Persistence per article to pick up exactly where you left off (This might become out of scope)

8) Firefox Extension (Or cross-browser)

This is the least researched epic in this document. I aim to have at least a single browser-supported extension, ideally cross-browser, but they have significant semantic differences in their APIs.

- The extension will allow users to authenticate by entering a generated code on the extension - inside the special screen on the app
- When authenticated, the extension will have 2 actions: save URL with the URL box and save the current page.

9) VPS setup and Play Store

System deployment. This will require IaC setup for provisioning infrastructure and a production Docker Compose setup for running the app on the VPS. The time is limited, so I will try to make the setup as production-ready and reproducible as time allows.

- I will use a simple OpenTofu (A logical pick, since I am a maintainer of it) for provisioning infrastructure on the Hetzner platform (Hetzner is chosen for cost-efficiency)
- To start, a single medium-sized VPS should work, but if time allow, I want to split the architecture for more reliability (at least the data node with replication)

This part also involves publishing the app. For the score of this project, I will be targeting the Android platform (extra: and binary releases for desktop).

I need to make a Google Developers account for publishing the app on the Store. Since this process involves a few legal matters that might stretch in time, it's hard to promise a Play Store published app by the end of this project.

Although the Google Developers account will be required to allow Google social authentication on Android Platforms.

10) Google Social Sign In

Requires a Google Developers account and a Kratos setup to make the native Google sign-in work on Android.

11) UI/UX polish

This is not my strong side, I admit. I will recruit help and feedback from friends to make the UX nice and easy to use, and have it coherent under a design system. This also includes choosing the color palette, gap/margin sizing used throughout the app, and so on.

12) Extras

This epic includes optional features and add-ons that would be awesome to have, but the time will not allow all of them to be included. If I have time in the last weeks, I will pick one or two features from the list below and add them to the app.

- RSS subscriptions and feed
- Homepage setup
- Article listening queue
- Text highlighting and bookmarks
- Text search in your saved articles

13) Buffer epic

For the finishing touches.

- Additional Testing
- General documentation
- Demo and presentation prep

Below is the Gantt Chart for roughly aligning the epics above on the 12 week timeline.